

1

Властивості молекул

Після обчислень електронної структури молекули природним кроком є обчислення її спостережуваних властивостей: коливальних спектрів, оптичних переходів, електричних та магнітних характеристик. PySCF надає потужні інструменти для розрахунку цих властивостей на рівні теорії Хартрі-Фока та пост-ХФ методів.

У попередніх розділах ви навчилися виконувати основні квантово-хімічні розрахунки: оптимізувати геометрію, обчислювати енергії, аналізувати хвильові функції. Тепер настав час перейти до **обчислення молекулярних властивостей** — фізичних величин, які можна безпосередньо порівняти з експериментом.

Що таке молекулярні властивості?

Молекулярні властивості — це спостережувані величини, які характеризують поведінку молекули у зовнішніх полях або при взаємодії з випромінюванням. На відміну від енергії чи геометрії, властивості безпосередньо вимірюються експериментально:

- **Коливальні спектри (ІЧ, Раман):** частоти, інтенсивності.
- **Електронні спектри (УФ-видимі):** довжини хвиль, сили осциляторів.
- **Електричні властивості:** дипольний момент, поляризованість.
- **Магнітні властивості:** ЯМР зсуви, g-тензор, константи спін-спінової взаємодії.
- **Оптична активність:** круговий дихроїзм, оптичне обертання.

Головна перевага обчислення властивостей: можна безпосередньо порівняти теорію з експериментом та перевірити якість обчислень!

Встановлення додаткових модулів PySCF

Базова інсталяція PySCF містить функціонал для розрахунку енергій, градієнтів та оптимізації геометрії. Для обчислення спектроскопічних та магнітних властивостей потрібні **додаткові модулі**.

Основні модулі для властивостей

1. **pyscf-properties** — головний модуль для властивостей:

```
pip install pyscf-properties
```

Цей модуль включає:

- `pyscf.prop.nmr` — ЯМР константи екранування
- `pyscf.prop.esr` — ЕПР g-тензор, константи надтонкої взаємодії
- `pyscf.prop.polarizability` — поляризованість, гіперполяризованості
- `pyscf.prop.magnetizability` — магнітна сприйнятливість

2. **pyscf-geomopt** — для оптимізації геометрії:

```
pip install pyscf-geomopt
```

3. **geometric** — покращена оптимізація (рекомендується):

```
pip install geometric
```

Перевірка встановлення

Після інсталяції перевірте, чи працюють модулі:

```
import pyscf
from pyscf import gto, scf
from pyscf.prop import nmr, esr # для магнітних властивостей
from pyscf.hessian import thermo # для коливальних властивостей
from pyscf import tddft # для електронних збуджень

print("PySCF версія:", pyscf.__version__)
print("Модулі успішно імпортовані!")
```

Якщо імпорт проходить без помилок — все готово для роботи!

Альтернатива: встановлення з conda

Для користувачів Anaconda/Miniconda:

```
conda install -c pyscf pyscf
conda install -c conda-forge geometric
pip install pyscf-properties # properties поки немає в conda
```

Концептуальна основа: теорія відгуку

Теорія відгуку (response theory) є потужним інструментом для обчислення молекулярних властивостей у квантовій механіці. Її суть полягає в аналізі реакції системи на зовнішні збурення: ми застосовуємо слабе зовнішнє поле (або збурення) і спостерігаємо, як змінюється енергія, хвильова функція чи інші observable величини. Цей підхід дозволяє систематично обчислювати похідні енергії по параметрах збурення, які безпосередньо відповідають фізичним властивостям, таким як поляризованість, магнітні моменти чи спектроскопічні переходи.

Загалом, енергія системи в присутності збурення λ (де λ може бути будь-яким параметром, наприклад, електричним або магнітним полем чи геометрією молекули тощо) розкладається в ряд Тейлора:

$$E(\lambda) = E_0 + \left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0} \lambda + \frac{1}{2!} \left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0} \lambda^2 + \frac{1}{3!} \left. \frac{\partial^3 E}{\partial \lambda^3} \right|_{\lambda=0} \lambda^3 + \dots$$

Коефіцієнти при степенях λ є похідними енергії по збуренню в точці $\lambda = 0$ і визначають лінійний, квадратичний тощо відгук системи. Ці похідні інтерпретуються як фундаментальні властивості:

- Перша похідна $\left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0}$ — лінійний відгук (наприклад, дипольний момент).
- Друга похідна $\left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0}$ — квадратичний відгук (наприклад, поляризованість).
- Вищі похідні — нелінійні ефекти (гіперполяризованість тощо).

Загальний принцип: будь-яка молекулярна властивість, пов'язана з відгуком на збурення, є похідною енергії (або хвильової функції) по відповідному параметру.

Приклад: розклад енергії в електричному полі

Для ілюстрації розглянемо конкретний випадок статичного однорідного електричного поля $\mathbf{E}$. Енергія системи розкладається як:

$$E(\mathbf{E}) = E_0 - \mu \cdot \mathbf{E} - \frac{1}{2} \sum_{ij} \alpha_{ij} E_i E_j - \frac{1}{6} \sum_{ijk} \beta_{ijk} E_i E_j E_k + \dots$$

де:

- E_0 — енергія системи без поля.
- μ — статичний дипольний момент (перша похідна $-\left. \frac{\partial E}{\partial \mathbf{E}} \right|_{\mathbf{E}=0}$).
- α_{ij} — тензор поляризованості (друга похідна $-\left. \frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{\mathbf{E}=0}$).
- β_{ijk} — тензор першої гіперполяризованості (третя похідна $-\left. \frac{\partial^3 E}{\partial E_i \partial E_j \partial E_k} \right|_{\mathbf{E}=0}$).

Цей розклад є частковим випадком загальної теорії відгуку, де $\lambda \equiv \mathbf{E}$. Аналогічно, теорія застосовується до магнітних полів (для магнітної сприйнятливості), геометричних змін (для сил і гессіанів) чи часозалежних збурень (для спектроскопії).

Загальний принцип:

властивість = похідна енергії по зовнішньому полю.

Методи обчислення похідних

1. Числові похідні (finite differences):

$$\alpha_{xx} = -\frac{\partial^2 E}{\partial E_x^2} \approx \frac{E(E_x) + E(-E_x) - 2E(0)}{\Delta E_x^2}$$

Плюси: просто, універсально

Мінуси: неточно, повільно, багато розрахунків

2. Аналітичні похідні:

$$\alpha_{ij} = -\left. \frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{E=0} = \text{складна формула через хвильову функцію}$$

Плюси: точно, швидко, один розрахунок

Мінуси: складна реалізація

3. Coupled-Perturbed методи (CP-HF, CP-KS):

Розв'язують рівняння для змін орбіталей під дією поля. Використовується в PySCF для більшості властивостей.

Рекомендація: завжди використовуйте аналітичні похідні, коли вони доступні!

Gauge-invariance для магнітних властивостей

Магнітні властивості (ЯМР, магнітна сприйнятливність) мають специфічну проблему: результат залежить від вибору **gauge** (калібрування) векторного потенціалу.

Проблема gauge origin

Магнітне поле $\mathbf{B}$ можна записати через векторний потенціал:

$$\mathbf{B} = \nabla \times \mathbf{A}$$

Але $\mathbf{A}$ не унікальний! Калібрувальне перетворення $\mathbf{A} \rightarrow \mathbf{A} + \nabla \chi$ не змінює $\mathbf{B}$.

Для скінченного базису результат залежить від вибору початку координат для $\mathbf{A}$ — це нефізично!

Рішення: GIAO (Gauge-Independent Atomic Orbitals)

У PySCF використовується метод GIAO (також називається IGLO, CSGT):

- Кожна атомна орбіталь має своє локальне калібрування.
- Результати не залежать від вибору початку координат.
- Швидка збіжність з розміром базису.
- **Стандарт** для розрахунків ЯМР властивостей.

В PySCF це працює автоматично — не потрібно нічого додатково налаштовувати!

```
from pyscf.prop import nmr

# GIAO використовується автоматично
nmr_calc = nmr.RHF(mf)
shielding = nmr_calc.kernel() # gauge-independent результат!
```

Базиси для розрахунку властивостей

Важливо: базиси для енергій $\neq$ базиси для властивостей!

Загальні вимоги

1. Дифузні функції (aug-, d-aug-):

Критично важливі для:

- Поляризовностей (опис деформації електронної густини)
- Магнітних властивостей (струми далеко від ядер)
- Анізотропних властивостей

Базиси: aug-cc-pVDZ, aug-cc-pVTZ, 6-311++G**

2. Високий angular momentum (поляризаційні функції):

Для точних властивостей потрібні d , f , іноді g функції.

3. Tight functions для ЯМР:

ЯМР екранування чутливе до густини на ядрі $\rho(\mathbf{R}_A)$. Спеціальні базиси: pcS-n, pcJ-n

Спеціалізовані базиси

Властивість	Рекомендований базис	Коментар
ІЧ частоти	6-31G*, 6-311G**	Невеликі базиси ОК
УФ-видимі спектри	aug-cc-pVDZ/TZ	Дифузні важливі
Поляризованість	aug-cc-pVDZ	Обов'язково aug-
ЯМР екранування	pcS-2, aug-cc-pVTZ	Tight + diffuse
J-константи	pcJ-2, pcJ-3	Спеціальний базис
ЕПР (g-тензор, HFC)	EPR-II, EPR-III	Tight s-функції

Практичні поради

- Для початку: **aug-cc-pVDZ** — універсальний вибір
- Для точних результатів: **aug-cc-pVTZ**
- Для великих систем: **6-311+G(2d,p)** — компроміс
- Для ЯМР: спеціалізовані базиси **pcS-n** кращі за aug-cc-pVnZ

Структура розділу

Розділ організовано за типами властивостей:

1. **Коливальні властивості (Секція 1.1):**
 - Гессіан та нормальні моди
 - ІЧ спектри (інтенсивності, правила відбору)
 - Раман спектри
 - Термохімія та ZPE
2. **Електронні властивості (Секція ??):**
 - Електронні збудження (TD-DFT)
 - УФ-видимі спектри
 - Флуоресценція та фосфоресценція
3. **Електричні властивості (Секція ??):**
 - Дипольний момент
 - Поляризованість (статична та динамічна)
 - Гіперполяризованості
 - Електростатичний потенціал
4. **Магнітні властивості (Секція ??):**
 - Магнітна сприйнятливості
 - ЯМР константи екранування та J-константи
 - ЕПР властивості (g-тензор, HFC)
 - Оптична активність (ORD, CD)

Практичні рекомендації перед початком

Типовий workflow розрахунку властивостей

1. Оптимізуйте геометрію:

```
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

mol = gto.M(atom='...', basis='6-31g*')
mf = scf.RHF(mol).run()
mol_eq = optimize(mf) # оптимізована геометрія
```

2. Перерахуйте з кращим базисом:

```
mol_prop = gto.M(atom=mol_eq.atom, basis='aug-cc-pvdz')
mf_prop = scf.RHF(mol_prop).run()
```

3. Обчисліть властивість:

```
from pyscf.prop import nmr
nmr_calc = nmr.RHF(mf_prop)
shielding = nmr_calc.kernel()
```

4. Проаналізуйте результат!

Типові помилки початківців

- Розрахунок властивостей без оптимізації геометрії
- Використання малих базисів (6-31G) для поляризовностей
- Забування про дифузні функції для анізотропних властивостей
- Розрахунок ЯМР без gauge-independent методу
- Порівняння абсолютних значень замість зсувів (для ЯМР)

Контрольний чек-лист

Перед розрахунком властивості перевірте:

- Геометрія оптимізована? (немає уявних частот)
- Базис підходить для цієї властивості?
- Метод підходить? (HF завищує поляризованість, потрібен DFT/MP2)
- Для магнітних: gauge-independent метод?
- SCF збігся? (check `mf.converged`)
- Для радикалів: перевірили $\langle S^2 \rangle$?

Корисні ресурси

- Документація PySCF: <https://pyscf.org/user.html>
- PySCF properties: <https://github.com/pyscf/properties>
- Basis Set Exchange: <https://www.basissetexchange.org/>
- NIST WebBook: експериментальні дані для порівняння

1.1. Коливальні властивості молекул

Коливальна спектроскопія — один із найпотужніших методів ідентифікації молекул та вивчення їх структури. Інфрачервоні (ІЧ) та Раман спектри надають інформацію про коливальні моди, силові константи зв'язків, та симетрію молекули.

Частоти в різних одиницях:

Одиниця	Множник	Використання
см ⁻¹	1	ІЧ спектроскопія
Гц	2.998×10^{10}	СІ одиниці
еВ	1.240×10^{-4}	Електронна спектроскопія
Хартрі	4.556×10^{-6}	Атомні одиниці

1.1.1. Гесіан та нормальні моди

Коливання атомів у молекулі відбуваються поблизу рівноважної геометрії $\mathbf{R}_0$. Для малих відхилень потенційна енергія системи можна апроксимувати розкладом у ряд Тейлора до другого порядку:

$$E(\mathbf{R}) \approx E(\mathbf{R}_0) + \frac{1}{2} \sum_{ij} H_{ij} \Delta R_i \Delta R_j,$$

де

$$H_{ij} = \left. \frac{\partial^2 E}{\partial R_i \partial R_j} \right|_{\mathbf{R}_0}$$

— елементи *гесіану* (матриці других похідних енергії за декартовими координатами ядер).

Гесіан має розмірність $3N \times 3N$ для молекули з N атомів і містить повну інформацію про гармонічні коливальні властивості системи.

Матриця силових констант

Гесіан також називають **матрицею силових констант**, оскільки його елементи відображають реакцію потенціалу на малі зміщення ядер:

$$H_{ij} = \left. \frac{\partial^2 E}{\partial R_i \partial R_j} \right|_{\mathbf{R}_0}$$

Фізичний зміст елементів гесіану:

- *Діагональні елементи* — характеризують жорсткість потенціалу вздовж окремих координат;
- *Недіагональні елементи* — описують зв'язок між зміщеннями різних атомів;
- *Власні значення* ω^2 — квадрати коливальних частот;
- *Власні вектори* — напрямки нормальних мод.

Таким чином, діагоналізація гесіану дозволяє отримати нормальні координати та спектр гармонічних частот, що повністю характеризують коливальну поведінку молекули поблизу рівноваги.

Нормальні коливання

Для переходу від гесіану $\mathbf{H}$ у декартових координатах до фізично осмислених частот враховують маси атомів. Вводять **мас-зважений гесіан**:

$$\mathbf{F} = \mathbf{M}^{-1/2} \mathbf{H} \mathbf{M}^{-1/2},$$

де $\mathbf{M}$ — діагональна матриця атомних мас.

Далі розв'язують задачу на власні значення:

$$\mathbf{F} \mathbf{Q} = \omega^2 \mathbf{Q},$$

де ω — коливальні частоти, а $\mathbf{Q}$ — вектори нормальних координат.

Отримані власні значення ω_i^2 відповідають:

- $3N - 6$ коливальним модам для нелінійної молекули;
- $3N - 5$ — для лінійної;
- 3 трансляційним і 3 (або 2) обертальним модам з $\omega = 0$.

Зв'язок координат Q_k та $\mathbf{R}$ Нормальні координати Q_k є лінійними комбінаціями атомних зміщень $\Delta \mathbf{R}$:

$$Q_k = \sum_{i=1}^{3N} \sqrt{m_i} l_{ik} \Delta R_i$$

або у матричній формі:

$$\mathbf{Q} = \mathbf{L}^T \mathbf{M}^{1/2} \Delta \mathbf{R},$$

де:

- $\mathbf{M}$ — діагональна матриця мас атомів,
- $\mathbf{L}$ — матриця власних векторів (нормальних мод),
- $\Delta \mathbf{R} = \mathbf{R} - \mathbf{R}_0$ — відхилення координат від рівноваги.

Звідси обернене перетворення:

$$\Delta \mathbf{R} = \mathbf{M}^{-1/2} \mathbf{L} \mathbf{Q}_k.$$

Саме цю формулу використовують у чисельному диференціюванні:

$$\mathbf{R}^{(\pm)} = \mathbf{R}_0 \pm h \mathbf{M}^{-1/2} \mathbf{L}_k,$$

що зв'язує крок h у просторі нормальних координат Q_k із фізичними зміщеннями атомів $\mathbf{R}$.

Масштабування частот

Розраховані гармонічні частоти $\nu_{\text{гарм}}$ систематично завищені відносно експериментальних $\nu_{\text{експ}}$ через спрощення моделі потенціальної поверхні та обмеження методів:

- **Гармонічне наближення:** реальний потенціал ангармонічний, тому енергія коливань і частоти менші.
- **Нестача електронної кореляції:** метод Хартрі–Фока завищує жорсткість зв'язків.
- **Обмежений базис:** малий базис перебільшує градієнти енергії.

Масштабування λ емпірично компенсує ангармонічність і методичні похибки. Емпіричне виправлення:

$$\nu_{\text{корект}} = \lambda \cdot \nu_{\text{гарм}}$$

Типові масштабувальні фактори:

Метод/базис	λ	Коментар
HF/6-31G*	0.8929	Сильно завищує
B3LYP/6-31G*	0.9613	Універсальний
B3LYP/6-311+G(2d,p)	0.9679	Кращий базис
MP2/6-31G*	0.9434	Для точних розрахунків

Джерело: NIST Computational Chemistry Comparison and Benchmark Database

1.1.2. Обчислення гесіану

PySCF може обчислювати гесіан аналітично або чисельно:

`h2o_hessian.py`

```
# =====
# h2o_hessian.py
# Обчислення гесіану чисельним та аналітичним способами
# Їх порівняння
# =====

import time

import numpy as np
from pyscf import gto, scf
from pyscf.hessian import rhf as rhf_hess

# Створення молекули води
mol = gto.M(
    atom="""
O  0.0000  0.0000  0.1173
H  0.0000  0.7572 -0.4692
H  0.0000 -0.7572 -0.4692
""",
    basis="6-31g",
    unit="angstrom",
)

print("=" * 60)
print("Обчислення Гесіану для молекули води (H2O)")
print("=" * 60)
print(f"\nБазис: {mol.basis}")
```

```
print(f"Кількість атомів: {mol.natm}")
print(f"Розмір Гесіану: {3 * mol.natm} x {3 * mol.natm}")

# Виконання SCF розрахунку
print("\n--- Розрахунок SCF ---")
mf = scf.RHF(mol)
mf.kernel()
print(f"Енергія SCF: {mf.e_tot:.10f} Ha")

# =====
# 1. ЧИСЕЛЬНИЙ ГЕСІАН (числове диференціювання градієнта)
# =====
print("\n" + "=" * 60)
print("1. ЧИСЕЛЬНИЙ ГЕСІАН")
print("=" * 60)
print("Обчислюємо через скінченні різниці градієнтів...")

start_time = time.time()

# Крок для числового диференціювання
step = 0.001 # в атомних одиницях (Bohr)

natm = mol.natm
hess_numerical = np.zeros((natm, natm, 3, 3))

# Обчислюємо градієнт для кожного зміщення
for atom_i in range(natm):
    for coord_i in range(3):
        # Позитивне зміщення
        coords_pos = mol.atom_coords().copy()
        coords_pos[atom_i, coord_i] += step

        mol_pos = gto.M(
            atom=[mol.atom_symbol(i), coords_pos[i]] for i in range(natm)],
            basis=mol.basis,
            unit="Bohr",
        )
        mf_pos = scf.RHF(mol_pos)
        mf_pos.verbose = 0
        mf_pos.kernel()
        grad_pos = mf_pos.Gradients().kernel()

        # Негативне зміщення
        coords_neg = mol.atom_coords().copy()
        coords_neg[atom_i, coord_i] -= step

        mol_neg = gto.M(
            atom=[mol.atom_symbol(i), coords_neg[i]] for i in range(natm)],
            basis=mol.basis,
            unit="Bohr",
        )
        mf_neg = scf.RHF(mol_neg)
        mf_neg.verbose = 0
        mf_neg.kernel()
        grad_neg = mf_neg.Gradients().kernel()

        # Числова похідна:  $d^2E/dx_i dx_j = (dE/dx_j|_{x_i+h} - dE/dx_j|_{x_i-h}) / 2h$ 
        hess_numerical[:, atom_i, :, coord_i] = (grad_pos - grad_neg) / (2 * step)

numerical_time = time.time() - start_time

print(f"\nЧас обчислення: {numerical_time:.2f} c")
print(f"Форма Гесіану: {hess_numerical.shape}")
```

```

# Перетворюємо в 2D для виводу
hess_num_2d = hess_numerical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)
print("\nЧисельний Гесіан (перші 6x6 елементів):")
print(hess_num_2d[:6, :6])

# =====
# 2. АНАЛІТИЧНИЙ ГЕСІАН (через аналітичні похідні)
# =====
print("\n" + "=" * 60)
print("2. АНАЛІТИЧНИЙ ГЕСІАН")
print("=" * 60)

start_time = time.time()

# Аналітичний Гесіан через pyscf.hessian.rhf
hess_analytical = rhf_hess.Hessian(mf).kernel()

analytical_time = time.time() - start_time

print(f"\nЧас обчислення: {analytical_time:.2f} с")
print(f"Форма Гесіану: {hess_analytical.shape}")

# Перетворюємо в 2D для виводу
hess_anal_2d = hess_analytical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)
print("\nАналітичний Гесіан (перші 6x6 елементів):")
print(hess_anal_2d[:6, :6])

# =====
# ПОРІВНЯННЯ РЕЗУЛЬТАТІВ
# =====
print("\n" + "=" * 60)
print("ПОРІВНЯННЯ РЕЗУЛЬТАТІВ")
print("=" * 60)

difference = hess_num_2d - hess_anal_2d
max_diff = np.max(np.abs(difference))
mean_diff = np.mean(np.abs(difference))

print(f"\nМаксимальна різниця: {max_diff:.2e}")
print(f"Середня абсолютна різниця: {mean_diff:.2e}")
print(f"\nПрискорення (аналітичний швидший у): {numerical_time / analytical_time:.2f}x")

print("\nМатриця різниць (перші 6x6 елементів):")
print(difference[:6, :6])

# =====
# ВЛАСНІ ЗНАЧЕННЯ (частоти коливань)
# =====
print("\n" + "=" * 60)
print("ВЛАСНІ ЗНАЧЕННЯ ГЕСІАНУ (з аналітичного)")
print("=" * 60)

# Перетворюємо Гесіан з форми (natm, natm, 3, 3) в (3*natm, 3*natm)
hess_2d = hess_analytical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)

# Масово-зважений Гесіан для частот
mass = np.array([mol.atom_mass_list()[i] for i in range(natm)])
mass_matrix = np.repeat(mass, 3)
mass_weighted_hess = hess_2d / np.sqrt(mass_matrix[:, None] * mass_matrix[None, :])

eigenvalues, eigenvectors = np.linalg.eigh(mass_weighted_hess)

# Конвертація в частоти (cm^-1)
# 1 Hartree/(amu*Bohr^2) * (a0/Bohr)^2 = omega^2

```

```
# omega = sqrt(eigenvalue) * conversion_factor
conversion = 5140.48 # приблизний коефіцієнт для Ha/(amu*angstrom^2) -> cm^-1

frequencies = np.sign(eigenvalues) * np.sqrt(np.abs(eigenvalues)) * conversion

print("\nЧастоти коливань (см^-1):")
print("(перші 6 значень мають бути близькі до нуля - трансляції та обертання)")
for i, freq in enumerate(frequencies):
    mode_type = "трансляція/обертання" if i < 6 else "коливання"
    print(f"  Мода {i + 1}: {freq:10.2f} см^-1 ({mode_type})")

print("\n" + "=" * 60)
print("ПРИМІТКИ:")
print("=" * 60)
print("1. Чисельний метод: диференціювання градієнтів методом центральних різниць")
print("2. Аналітичний метод: pyscf.hessian.rhf.Hessian - точні другі похідні")
print("3. Аналітичний метод набагато швидший та точніший!")
print("=" * 60)
```

Програма обчислює матрицю Гесіану (матрицю других похідних енергії) для молекули води (H_2O) двома методами: чисельним та аналітичним.

Коментар

У програмному пакеті PySCF гесіан зберігається не як звичайна двовимірна матриця розмірності $3N \times 3N$, а у вигляді чотиривимірного тензора

$$H_{AB,ij} = \frac{\partial^2 E}{\partial R_{A,i} \partial R_{B,j}},$$

де A, B — індекси атомів ($1, \dots, N$), а $i, j \in \{x, y, z\}$ — напрямки декартових координат. Відповідно, внутрішня структура масиву має вигляд

$$H \in \mathbb{R}^{N \times N \times 3 \times 3}.$$

Щоб отримати стандартну матричну форму $3N \times 3N$, необхідно спершу переставити осі, розмістивши напрямки координат поруч із відповідними атомами:

$$H'_{Ai,Bj} = H_{AB,ij}.$$

Це виконується операцією

$$H' = H.\text{transpose}(0, 2, 1, 3).\text{reshape}(3N, 3N).$$

Така трансформація формує коректну блочну структуру з блоками 3×3 , яка відповідає фізичному гесіану молекули у декартових координатах.

Чисельний Гесіан

Чисельний Гесіан обчислюється через скінченні різниці методом центральних різниць:

1. **Зміщення координат:** Кожна координата x_i кожного атома зміщується на величину $\pm h$ (крок диференціювання):

$$x_i^{(\pm)} = x_i \pm h \quad (1.1)$$

де $h = 0.001$ Bohr (атомні одиниці).

2. **Обчислення градієнтів:** Для кожного зміщення обчислюється градієнт енергії:

$$\nabla E(x_i^{(+)}) = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial x_2}, \dots, \frac{\partial E}{\partial x_N} \right)_{x_i=x_i+h} \quad (1.2)$$

$$\nabla E(x_i^{(-)}) = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial x_2}, \dots, \frac{\partial E}{\partial x_N} \right)_{x_i=x_i-h} \quad (1.3)$$

3. **Обчислення Гесіану:** Елементи матриці Гесіану обчислюються як:

$$H_{ij} = \frac{\partial^2 E}{\partial x_i \partial x_j} = \frac{\nabla E_j(x_i^{(+)}) - \nabla E_j(x_i^{(-)})}{2h} \quad (1.4)$$

Формула центральних різниць:

$$H_{ij} = \frac{g_j(x_i + h) - g_j(x_i - h)}{2h} \quad (1.5)$$

де $g_j = \frac{\partial E}{\partial x_j}$ — компонента градієнта.

Аналітичний Гесіан

Аналітичний Гесіан обчислюється через точні другі похідні енергії, використовуючи модуль PySCF:

$$H_{ij} = \frac{\partial^2 E}{\partial x_i \partial x_j} \quad (1.6)$$

Метод використовує `pyscf.hessian.rhf.Hessian(mf).kernel()` для прямого обчислення всіх елементів матриці Гесіану без числового диференціювання.

Обчислювальна складність

- **Чисельний метод:** Потребує $2 \times 3N$ SCF розрахунків, де N — кількість атомів.
 - Для H_2O ($N = 3$): $2 \times 3 \times 3 = 18$ SCF розрахунків.
 - Час виконання: $\sim 10 - 30$ секунд.
- **Аналітичний метод:** Один розрахунок з аналітичними похідними.
 - Час виконання: $\sim 1 - 5$ секунд.
 - **Прискорення:** у $5 - 20$ разів швидше.

Точність

- **Аналітичний метод:** Точніший, оскільки використовує точні похідні без похибок дискретизації

- **Чисельний метод:** Точність залежить від кроку h :

$$\text{Похибка} \sim O(h^2) + \frac{\varepsilon}{h} \quad (1.7)$$

де ε — машинна точність

- **Типова різниця:** Максимальна абсолютна різниця $\sim 10^{-5}$ – 10^{-7} Ha/Bohr².

Коливальні частоти

З матриці Гесіану можна отримати коливальні частоти молекули:

$$\omega_i = \sqrt{\lambda_i} \times 5140.48, \quad (1.8)$$

де 5140.48 — це числовий множник, який переводить частоти з атомних одиниць у cm^{-1} , а λ_i — власні значення масово-зваженого Гесіану:

$$\tilde{H}_{ij} = \frac{H_{ij}}{\sqrt{m_i m_j}} \quad (1.9)$$

Очікувані частоти для H_2O :

- Перші 6 мод: $\approx 0 \text{ cm}^{-1}$ (трансляції та обертання)
- Симетричне валентне коливання: $\approx 3600 \text{ cm}^{-1}$
- Деформаційне коливання: $\approx 1600 \text{ cm}^{-1}$
- Асиметричне валентне коливання: $\approx 3800 \text{ cm}^{-1}$

Для практичних розрахунків **аналітичний метод є кращим вибором**, оскільки він значно швидший та точніший. Чисельний метод корисний для перевірки результатів або коли аналітичні похідні недоступні.

1.1.3. Інфрачервоні (ІЧ) спектри

ІЧ-спектроскопія досліджує взаємодію молекул з інфрачервоним випромінюванням. Поглинання відбувається тоді, коли частота випромінювання збігається з частотою внутрішнього коливання молекули, тобто виконується умова резонансу:

$$h\nu = \Delta E_{\text{колив.}}$$

Ці переходи відповідають змінам *коливальних* рівнів енергії. Типові частоти для молекулярних коливань лежать у межах

$$400 - 4000 \text{ cm}^{-1},$$

що відповідає довжинам хвиль у мікрометровому діапазоні ($2.5 - 25 \mu\text{m}$).

ІЧ-спектр відображає лише ті *коливальні моди молекули*, які супроводжуються *зміною її дипольного моменту*.

Інтенсивності ІЧ поглинання

ІЧ інтенсивність пропорційна квадрату зміни дипольного моменту:

$$I_i \propto \left| \frac{d\mu}{dQ_i} \right|^2$$

де Q_i — нормальна координата i -ї моди, μ — дипольний момент.

Правило відбору:

Коливання є **ІЧ-активним** тільки якщо:

$$\left| \frac{\partial \mu}{\partial Q_k} \right| \neq 0 \quad (1.10)$$

Це означає, що дипольний момент молекули має змінюватися під час коливання. Симетрична молекула може мати *ІЧ-неактивні* моди.

Повний вектор похідної дипольного моменту по нормальним координатам Q_k :

$$\frac{\partial \mu}{\partial Q_k} = \left(\frac{\partial \mu_x}{\partial Q_k}, \frac{\partial \mu_y}{\partial Q_k}, \frac{\partial \mu_z}{\partial Q_k} \right) \quad (1.11)$$

Його норма:

$$\left| \frac{\partial \mu}{\partial Q_k} \right| = \sqrt{\left(\frac{\partial \mu_x}{\partial Q_k} \right)^2 + \left(\frac{\partial \mu_y}{\partial Q_k} \right)^2 + \left(\frac{\partial \mu_z}{\partial Q_k} \right)^2} \quad (1.12)$$

Інтенсивність обчислюється як:

$$I_k = K \times \left| \frac{\partial \mu}{\partial Q_k} \right|^2, \quad (1.13)$$

Дипольний момент в атомних одиницях (a.u. = $e \cdot a_0$):

$$1 \text{ a.u.} = 2.5418 \text{ Debye} \quad (1.14)$$

де e — заряд електрона, $a_0 = 0.529177 \text{ \AA}$ — радіус Бора.

Похідна в атомних одиницях має розмірність:

$$\left[\frac{\partial \mu}{\partial Q} \right] = \frac{e \cdot a_0}{a_0 \cdot \sqrt{\text{amu}}} = \frac{e}{\sqrt{\text{amu}}} \quad (1.15)$$

Отже:

$$I \text{ (кМ/моль)} = \frac{N_A \pi}{3c^2} \left| \frac{\partial \mu}{\partial Q} \right|^2 \quad (1.16)$$

де:

- $N_A = 6.022 \times 10^{23} \text{ моль}^{-1}$ — число Авогадро
- $c = 2.998 \times 10^{10} \text{ см/с}$ — швидкість світла

Після підстановки констант:

$$I = 974.8638 \times \left| \frac{\partial \mu}{\partial Q} \right|_{\text{a.u.}}^2 \quad (1.17)$$

1.1.4. Класифікація за інтенсивністю

Інтенсивність (км/моль)	Класифікація
$I > 100$	Сильна смуга
$10 < I \leq 100$	Середня смуга
$I \leq 10$	Слабка смуга

1.1.5. ІЧ спектр H_2O

Програма використовує **метод центральних різниць** для чисельного обчислення $\frac{\partial \mu}{\partial Q_k}$:

$$\frac{\partial \mu}{\partial Q_k} \approx \frac{\mu(Q_k + h) - \mu(Q_k - h)}{2h} \quad (1.18)$$

де $h = 0.001$ Bohr — крок диференціювання.

Алгоритм обчислення

1. Отримання нормальної моди:

З діагоналізації масово-зваженого Гесіану отримуємо власний вектор $\mathbf{L}_k$ (нормальну моду):

$$\mathbf{L}_k = (l_{1x}, l_{1y}, l_{1z}, l_{2x}, l_{2y}, l_{2z}, \dots, l_{Nx}, l_{Ny}, l_{Nz})^T \quad (1.19)$$

де N — кількість атомів.

2. Зміщення координат:

Зміщуємо координати всіх атомів вздовж нормальної моди:

$$\mathbf{R}^{(+)} = \mathbf{R}_0 + h \cdot \mathbf{L}_k \quad (1.20)$$

$$\mathbf{R}^{(-)} = \mathbf{R}_0 - h \cdot \mathbf{L}_k \quad (1.21)$$

3. SCF розрахунки:

Для кожної зміщеної геометрії виконуємо SCF розрахунок:

$$E^{(+)}, \mu^{(+)} = \text{SCF}(\mathbf{R}^{(+)}) \quad (1.22)$$

$$E^{(-)}, \mu^{(-)} = \text{SCF}(\mathbf{R}^{(-)}) \quad (1.23)$$

4. Обчислення похідної:

Чисельна похідна дипольного моменту:

$$\frac{\partial \mu}{\partial Q_k} = \frac{\mu^{(+)} - \mu^{(-)}}{2h} \quad (1.24)$$

h2o_ir_spectrum.py

```
# =====
# h2o_ir_spectrum.py
# Розрахунок ІЧ-спектру (частоти + інтенсивності)
# =====

import numpy as np
from pyscf import gto, scf
from pyscf.hessian import rhf as rhf_hess
from pyscf.hessian import thermo
import matplotlib.pyplot as plt

# =====
# Створення молекули H2O
# =====
mol = gto.M(
    atom="""
    O  0.0000  0.0000  0.1173
    H  0.0000  0.7572 -0.4692
    H  0.0000 -0.7572 -0.4692
    """,
    basis="6-31g",
    unit="angstrom",
)

print("=" * 70)
print("Розрахунок ІЧ-спектру H2O (частоти + інтенсивності)")
print("=" * 70)

# =====
# SCF розрахунок
# =====
print("\n[1/4] SCF розрахунок...")
mf = scf.RHF(mol)
mf.kernel()
print(f"Енергія: {mf.e_tot:.8f} Ha")

# =====
# Обчислення Гесіану
# =====
print("\n[2/4] Обчислення Гесіану...")
hess = mf.Hessian().kernel()

# =====
# Аналіз нормальних мод
# =====
print("\n[3/4] Аналіз нормальних мод...")
freq_info = thermo.harmonic_analysis(mol, hess)

frequencies = freq_info['freq_wavenumber'] # в см-1
normal_modes = freq_info['norm_mode'] # власні вектори

# =====
# Обчислення інтенсивностей ІЧ
# =====
print("\n[4/4] Обчислення інтенсивностей ІЧ...")

# Крок для числового диференціювання дипольного моменту
step = 0.001 # Bohr

natm = mol.natm
intensities = []

for mode_idx in range(len(frequencies)):
```

```

# Нормальна мода (3*natm вектор)
mode = normal_modes[:, mode_idx].reshape(natm, 3)

# Зміщення вздовж нормальної моди
coords_orig = mol.atom_coords() # в Bohr

# Позитивне зміщення
coords_pos = coords_orig + step * mode
mol_pos = gto.M(
    atom=[[mol.atom_symbol(i), coords_pos[i]] for i in range(natm)],
    basis=mol.basis,
    unit='Bohr'
)
mf_pos = scf.RHF(mol_pos)
mf_pos.verbose = 0
mf_pos.kernel()
dip_pos = mf_pos.dip_moment(unit='AU') # атомні одиниці

# Негативне зміщення
coords_neg = coords_orig - step * mode
mol_neg = gto.M(
    atom=[[mol.atom_symbol(i), coords_neg[i]] for i in range(natm)],
    basis=mol.basis,
    unit='Bohr'
)
mf_neg = scf.RHF(mol_neg)
mf_neg.verbose = 0
mf_neg.kernel()
dip_neg = mf_neg.dip_moment(unit='AU')

# Похідна дипольного моменту: dμ/dQ
dip_derivative = (dip_pos - dip_neg) / (2 * step)

# Інтенсивність: I ∝ |dμ/dQ|^2
# В одиницях: (Debye/Angstrom)^2 -> km/mol
# Константа: 974.8638 для конвертації
intensity = np.linalg.norm(dip_derivative)**2 * 974.8638
intensities.append(intensity)

intensities = np.array(intensities)

# =====
# Виведення результатів
# =====
print("\n" + "=" * 70)
print("ІЧ-СПЕКТР H2O")
print("=" * 70)
print(f"{'Мода':<8} {'ν (см⁻¹)':<15} {'Інтенсивність (km/mol)':<25} {'Тип'}")
print("-" * 70)

for i, (freq, intens) in enumerate(zip(frequencies, intensities)):
    if freq > 100: # Тільки справжні коливання (не трансляції/обертання)
        mode_type = "сильна" if intens > 100 else ("середня" if intens > 10 else "слабка")
        print(f"{i+1:<8} {freq:>12.1f} {intens:>20.1f} {mode_type}")

# =====
# Візуалізація спектру
# =====
print("\n[Візуалізація] Побудова ІЧ-спектру...")

# Фільтруємо тільки справжні коливання
real_modes = frequencies > 100
freq_plot = frequencies[real_modes]
intens_plot = intensities[real_modes]

```

```

# Створюємо спектр з гауссовим розширенням
x = np.linspace(0, 4500, 2000)
y = np.zeros_like(x)
fwhm = 20 # Full Width at Half Maximum (cm-1)

for freq, intens in zip(freq_plot, intens_plot):
    sigma = fwhm / (2 * np.sqrt(2 * np.log(2)))
    y += intens * np.exp(-((x - freq)**2) / (2 * sigma**2))

# Побудова графіку
plt.figure(figsize=(12, 6))
plt.plot(x, y, 'b-', linewidth=1.5)
plt.fill_between(x, y, alpha=0.3)

# Додаємо вертикальні лінії для піків
for freq, intens in zip(freq_plot, intens_plot):
    plt.axvline(freq, color='r', linestyle='--', alpha=0.5, linewidth=0.8)
    plt.text(freq, intens * 1.05, f'{freq:.0f}',
             rotation=90, va='bottom', ha='right', fontsize=8)

plt.xlabel('Хвильове число (см-1)', fontsize=12)
plt.ylabel('Інтенсивність (km/mol)', fontsize=12)
plt.title('ІЧ-спектр H2O (RHF/6-31G)', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.xlim(0, 4500)
plt.ylim(0, max(y) * 1.2)

plt.tight_layout()
plt.savefig('h2o_ir_spectrum.png', dpi=300, bbox_inches='tight')
print("Спектр збережено в 'h2o_ir_spectrum.png'")
plt.show()

# =====
# Інтерпретація мод
# =====
print("\n" + "=" * 70)
print("ІНТЕРПРЕТАЦІЯ КОЛИВАЛЬНИХ МОД")
print("=" * 70)
print("""
Для H2O очікуються 3 коливальні моди:

1. Симетричне валентне (ν1): ~3600 см-1
   - Обидва H-O зв'язки розтягуються синхронно
   - Середня інтенсивність

2. Деформаційне (ν2): ~1600 см-1
   - Зміна кута H-O-H
   - Найсильніша інтенсивність

3. Асиметричне валентне (ν3): ~3800 см-1
   - Один H-O зв'язок розтягується, інший стискається
   - Сильна інтенсивність

Примітка: Абсолютні значення можуть відрізнятися від експерименту
через наближення (базис 6-31G, RHF метод).
""")

print("=" * 70)
print("Розрахунок завершено!")
print("=" * 70)

```

1.1.6. Результати для H₂O

Нормальні моди води

Молекула H₂O має $3N - 6 = 3$ коливальні моди:

Мода	ν (см ⁻¹)	I (км/моль)	Тип
1	1828.9	1.4	Слабка (деформаційна)
2	3906.0	102.3	Сильна (симетричне валентне)
3	4001.1	138.6	Сильна (асиметричне валентне)

Примітка. Усі три моди молекули H₂O є ІЧ-активними (група симетрії C_{2v}). Найсильніше поглинання відповідає валентним коливанням, що зумовлюють інтенсивну ІЧ-смугу пари води в атмосфері (важливий внесок у парниковий ефект).

Обчислювальна складність

Для молекули з M коливальними модами (без трансляцій/обертань):

$$N_{\text{SCF}} = 1 + 2M \quad (1.25)$$

- 1 — початковий SCF для рівноважної геометрії
- $2M$ — по два SCF ($\pm h$) для кожної моди

Для H₂O: $N_{\text{SCF}} = 1 + 2 \times 3 = 7$ розрахунків.

Типовий час для H₂O/6-31G:

- Гесіан: ~ 1 – 2 с
- ІЧ інтенсивності: ~ 5 – 10 с
- Загальний час: ~ 6 – 12 с

Чисельні похибки

1. Вибір кроку h :

- Занадто великий h — похибка дискретизації $O(h^2)$
- Занадто малий h — похибка округлення $\sim \epsilon/h$
- Оптимальний $h \sim 10^{-3}$ Bohr

2. Збіжність SCF:

- Кожен SCF має свою похибку збіжності
- Критерій збіжності: 10^{-8} Ha

Методологічні обмеження

1. Гармонічне наближення:

- Використовується квадратична апроксимація енергії
- Ангармонізм не враховується
- Для високочастотних мод похибка може бути > 100 см⁻¹

2. Якість базису:

- 6-31G — мінімальний базис для якісних результатів
- Для точних інтенсивностей потрібні 6-311++G** або aug-cc-pVTZ

3. Рівень теорії:

- RHF не враховує електронну кореляцію
- DFT (B3LYP) дає кращі результати
- MP2/CCSD — найточніші, але дорогі

Покращення точності**1. Використовувати кращі базиси:**

```
basis = "6-311++G**" # або "aug-cc-pVTZ"
```

2. Використовувати DFT:

```
mf = dft.RKS(mol)
mf.xc = "B3LYP"
```

3. Масштабувати частоти:

Емпіричне масштабування для RHF/6-31G:

$$\nu_{\text{corr}} = 0.8929 \times \nu_{\text{calc}} \quad (1.26)$$

4. Враховувати ангармонізм:

Використовувати VPT2 (Vibrational Perturbation Theory)

Висновок Програма `h2o_ir_spectrum.py` надає повний розрахунок ІЧ-спектру, використовуючи:

- Аналітичний Гесіан для частот (швидко та точно)
- Чисельне диференціювання дипольного моменту для інтенсивностей
- Метод центральних різниць ($O(h^2)$ точність)

Результати є якісно правильними для розуміння коливальної структури молекули. Для кількісних порівнянь з експериментом потрібні:

- Кращі базисні набори
- Врахування електронної кореляції (DFT або пост-HF)
- Ангармонічні корекції
- Врахування ефектів середовища (розчинник)

1.1.7. Раманівський спектр

На відміну від ІЧ, активність у Раман-спектрі визначається поляризованістю α , тобто інтенсивність Раман розсіювання пропорційна зміні поляризованості:

$$I_i^{\text{Раман}} \propto \left| \frac{d\alpha_{ij}}{dQ_k} \right|^2$$

Інтенсивності Раман розсіювання

Правило відбору Раман:

- Раман-активна: $\frac{d\alpha}{dQ_i} \neq 0$.
- Часто доповнює ІЧ (правило взаємного виключення для центросиметричних).
- Для H_2O : всі три моди також Раман-активні

h2o_raman_activity.py

```
# =====
# h2o_raman_activity.py
# =====
from pyscf import gto, scf
from pyscf.prop.polarizability import rhf as pol_rhf
import numpy as np

mol = gto.M(
    atom="""
    O  0.0000  0.0000  0.1173
    H  0.0000  0.7572 -0.4692
    H  0.0000 -0.7572 -0.4692
    """,
    basis="6-31g",
    unit="angstrom",
)

print("Розрахунок Раман-активності H2O")
print("=" * 60)

# SCF
mf = scf.RHF(mol)
mf.kernel()

# Поляризовність
polar = pol_rhf.Polarizability(mf)
alpha = polar.polarizability()

print("\nСтатична поляризовність (au³):")
print(f"αxx = {alpha[0, 0]:.4f}")
print(f"αyy = {alpha[1, 1]:.4f}")
print(f"αzz = {alpha[2, 2]:.4f}")

alpha_mean = np.trace(alpha) / 3
print(f"\nСередня поляризовність: {alpha_mean:.4f} au³")
print(f"Середня поляризовність: {alpha_mean * 0.1482:.4f} Å³")

print("\nРаман-активні моди (якісно):")
print("- Симетричне розтягування OH: сильна")
print("- Деформація HON: середня")
print("- Асиметричне розтягування OH: слабка")
```

Типові активності для H_2O :

Мода	ν (см ⁻¹)	Раман-активність (Å ⁴ /amu)
ν_1	3657	1.8
ν_2	1595	0.4
ν_3	3756	3.2

Принцип взаємного виключення:

Для молекул з центром інверсії (наприклад, CO_2):

- Парні моди (g): Раман активні, ІЧ неактивні
- Непарні моди (u): ІЧ активні, Раман неактивні

Раман спектр бензену

`benzene_raman_spectrum.py`

```
# =====
# benzene_raman_spectrum.py
# Повний розрахунок Раман-спектру бензолу C6H6
# Включає: частоти, активності, деполаризаційні відношення
# =====

from pyscf import gto, scf, dft
from pyscf.hessian import thermo
from pyscf.prop import polarizability
import numpy as np
import matplotlib.pyplot as plt

print("=" * 70)
print("РАМАН-СПЕКТР БЕНЗОЛУ C6H6")
print("=" * 70)

# =====
# Крок 1: Визначення молекули бензолу
# =====
print("\nКрок 1: Створення молекули бензолу (D6h симетрія)")
print("-" * 70)

# Бензол з правильною гексагональною структурою
mol = gto.M(
    atom="""
C   1.3970   0.0000   0.0000
C   0.6985   1.2100   0.0000
C  -0.6985   1.2100   0.0000
C  -1.3970   0.0000   0.0000
C  -0.6985  -1.2100   0.0000
C   0.6985  -1.2100   0.0000
H   2.4810   0.0000   0.0000
H   1.2405   2.1486   0.0000
H  -1.2405   2.1486   0.0000
H  -2.4810   0.0000   0.0000
H  -1.2405  -2.1486   0.0000
H   1.2405  -2.1486   0.0000
""",
    basis="6-311g**",
    unit="angstrom",
    symmetry=True, # Використовуємо симетрію
)

print(f"Кількість атомів: {mol.natm}")
print(f"Точкова група: {mol.symmetry}")
print(f"Базис: {mol.basis}")

# =====
# Крок 2: SCF розрахунок
# =====
print("\nКрок 2: DFT розрахунок (B3LYP/6-311G**)")
print("-" * 70)

mf = dft.RKS(mol)
```



```

mf.xc = "b3lyp"
mf.verbose = 4
e_total = mf.kernel()

print(f"\nПовна енергія: {e_total:.8f} Ha")

# =====
# Крок 3: Обчислення гесіану та частот
# =====
print("\n" + "=" * 70)
print("Крок 3: Розрахунок коливальних частот")
print("=" * 70)

print("\nОбчислення гесіану (це може зайняти 2-3 хвилини)...")
hess = mf.Hessian()
h = hess.kernel()

# Аналіз частот
freq_info = thermo.harmonic_analysis(mol, h)
frequencies = freq_info['freq_wavenumber']
normal_modes = freq_info['norm_mode']

# Фільтруємо справжні коливання (>100 cm-1)
real_freqs_mask = frequencies > 100
real_freqs = frequencies[real_freqs_mask]
real_modes = normal_modes[:, real_freqs_mask]

print(f"\nЗнайдено {len(real_freqs)} коливальних мод")

# Виводимо частоти по групах
print("\nКоливальні частоти (cm-1):")
print("-" * 70)
for i, freq in enumerate(real_freqs, 1):
    print(f"v_{i:2d} = {freq:7.1f} cm-1")

# =====
# Крок 4: Статична поляризованість
# =====
print("\n" + "=" * 70)
print("Крок 4: Статична поляризованість α(0)")
print("=" * 70)

pol = polarizability.rks.Polarizability(mf)
alpha_static = pol.polarizability()

print("\nТензор поляризованості α (au3):")
print("      x      y      z")
for i, label in enumerate(['x', 'y', 'z']):
    print(f"{label} ", end="")
    for j in range(3):
        print(f"{alpha_static[i,j]:>10.4f} ", end="")
    print()

alpha_mean = np.trace(alpha_static) / 3
print(f"\nСередня поляризованість: ᾱ = {alpha_mean:.4f} au3")

# =====
# Крок 5: Похідні поляризованості (Раман-активність)
# =====
print("\n" + "=" * 70)
print("Крок 5: Обчислення Раман-активностей")
print("=" * 70)

print("\nОбчислення похідних поляризованості ∂α/∂Q...")

```

```

print("(Використовуємо чисельне диференціювання)")

# Параметри для чисельного диференціювання
delta = 0.01 # Зміщення в атомних одиницях (bohr)

raman_activities = []
depolarization_ratios = []

for mode_idx in range(len(real_freqs)):
    print(f"\rОбробка моди {mode_idx+1}/{len(real_freqs)}...", end="", flush=True)

    # Нормальна мода у декартових координатах
    mode = real_modes[:, mode_idx]

    # Зміщення вздовж нормальної моди
    coords_orig = mol.atom_coords()

    # +delta
    coords_plus = coords_orig + delta * mode.reshape(-1, 3)
    mol_plus = gto.M(
        atom=[(mol.atom_symbol(i), coords_plus[i]) for i in range(mol.natm)],
        basis=mol.basis,
        unit="bohr",
        symmetry=False,
        verbose=0,
    )
    mf_plus = dft.RKS(mol_plus)
    mf_plus.xc = mf.xc
    mf_plus.verbose = 0
    mf_plus.kernel()
    pol_plus = polarizability.rks.Polarizability(mf_plus)
    alpha_plus = pol_plus.polarizability()

    # -delta
    coords_minus = coords_orig - delta * mode.reshape(-1, 3)
    mol_minus = gto.M(
        atom=[(mol.atom_symbol(i), coords_minus[i]) for i in range(mol.natm)],
        basis=mol.basis,
        unit="bohr",
        symmetry=False,
        verbose=0,
    )
    mf_minus = dft.RKS(mol_minus)
    mf_minus.xc = mf.xc
    mf_minus.verbose = 0
    mf_minus.kernel()
    pol_minus = polarizability.rks.Polarizability(mf_minus)
    alpha_minus = pol_minus.polarizability()

    # Похідна:  $\partial\alpha/\partial Q \approx (\alpha(+\delta) - \alpha(-\delta)) / (2\delta)$ 
    dalpha_dQ = (alpha_plus - alpha_minus) / (2 * delta)

    # Раман-активність (середнє поле)
    #  $S = 45 * (\partial\alpha/\partial Q)^2 + 7 * (\partial\gamma/\partial Q)^2$ 
    # де  $\alpha$  - середня поляризованість,  $\gamma$  - анізотропія

    dalpha_mean = np.trace(dalpha_dQ) / 3

    # Анізотропія похідної
    dalpha_aniso_sq = (
        (dalpha_dQ[0,0] - dalpha_dQ[1,1])**2 +
        (dalpha_dQ[1,1] - dalpha_dQ[2,2])**2 +
        (dalpha_dQ[2,2] - dalpha_dQ[0,0])**2 +
        6 * (dalpha_dQ[0,1]**2 + dalpha_dQ[0,2]**2 + dalpha_dQ[1,2]**2)

```

```

) / 2

# Раман-активність ( $\text{\AA}^4/\text{amu}$ )
S = 45 * dalpha_mean**2 + 7 * dalpha_aniso_sq
raman_activities.append(S)

# Деполяризаційне відношення (для неполяризованого світла)
#  $\rho = 3\gamma^2 / (45\alpha^2 + 4\gamma^2)$ 
if 45 * dalpha_mean**2 + 4 * dalpha_aniso_sq > 1e-10:
    rho = 3 * dalpha_aniso_sq / (45 * dalpha_mean**2 + 4 * dalpha_aniso_sq)
else:
    rho = 0.0
depolarization_ratios.append(rho)

print("\n")

raman_activities = np.array(raman_activities)
depolarization_ratios = np.array(depolarization_ratios)

# =====
# Крок 6: Аналіз результатів
# =====
print("\n" + "=" * 70)
print("Крок 6: Аналіз Раман-спектру")
print("=" * 70)

print("\nРаман-активні моди бензолу:")
print("-" * 70)
print(f"{'№':<5} {'v (см $\text{\AA}^{-1}$ ):<12} {'Активність':<15} {'p':<10} {'Тип'}")
print("-" * 70)

# Класифікація мод за деполяризаційним відношенням
for i, (freq, activity, rho) in enumerate(
    zip(real_freqs, raman_activities, depolarization_ratios), 1
):
    # Класифікація
    if activity > 10: # Значуща активність
        if rho < 0.1:
            mode_type = "Повністю симетрична"
        elif 0.1 ≤ rho < 0.5:
            mode_type = "Симетрична"
        else:
            mode_type = "Деполяризована"

        print(f"{'i':<5} {'freq:>10.1f} {'activity:>12.2f} {'rho:>8.4f} {'mode_type}"))

# Найінтенсивніші моди
print("\nНайінтенсивніші Раман-лінії:")
print("-" * 70)
top_indices = np.argsort(raman_activities)[-5:][::-1]
for idx in top_indices:
    freq = real_freqs[idx]
    activity = raman_activities[idx]
    rho = depolarization_ratios[idx]
    print(f"v = {freq:7.1f} см $\text{\AA}^{-1}$ , S = {activity:8.2f}, p = {rho:.4f}")

# =====
# Крок 7: Візуалізація спектру
# =====
print("\n" + "=" * 70)
print("Крок 7: Побудова Раман-спектру")
print("=" * 70)

# Сітка частот

```

```

freq_grid = np.linspace(0, 3500, 3500)

# Параметр уширення (FWHM)
gamma = 10 #  $\text{cm}^{-1}$ 

def lorentzian(x, x0, gamma, intensity):
    """Лоренцева функція"""
    return intensity * (gamma/2)**2 / ((x - x0)**2 + (gamma/2)**2)

# Будуємо спектр
spectrum = np.zeros_like(freq_grid)
for freq, activity in zip(real_freqs, raman_activities):
    if activity > 1: # Тільки значущі
        spectrum += lorentzian(freq_grid, freq, gamma, activity)

# Візуалізація
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

# 1. Штрих-спектр
ax1 = axes[0]
for freq, activity in zip(real_freqs, raman_activities):
    if activity > 1:
        ax1.stem([freq], [activity], linefmt='b-', markerfmt='bo', basefmt=' ')
ax1.set_xlim(0, 3500)
ax1.set_ylabel('Раман-активність ( $\nu/\text{amu}$ )', fontsize=11)
ax1.set_title('Раман-спектр бензолу  $\text{C}_6\text{H}_6$  (штрих-спектр)', fontsize=13, fontweight='bold')
ax1.grid(alpha=0.3)

# 2. Уширений спектр
ax2 = axes[1]
ax2.plot(freq_grid, spectrum, 'b-', linewidth=2)
ax2.set_xlim(0, 3500)
ax2.set_ylabel('Інтенсивність (відн. од.)', fontsize=11)
ax2.set_title('Уширений спектр ( $\gamma=10 \text{ cm}^{-1}$ )', fontsize=13)
ax2.grid(alpha=0.3)

# Позначки характерних смуг
characteristic_peaks = [
    (992, "Дихання кільця ( $A_{1g}$ )"),
    (1606, "C=C розтяг ( $E_{2g}$ )"),
    (3064, "C-H розтяг ( $E_{2g}$ )"),
]
for freq, label in characteristic_peaks:
    if freq < 3500:
        ax2.axvline(freq, color='r', linestyle='--', alpha=0.5)
        ax2.text(freq, ax2.get_ylim()[1]*0.9, label,
                 rotation=90, verticalalignment='top', fontsize=9)

# 3. Деполяризаційні відношення
ax3 = axes[2]
colors = ['green' if rho < 0.1 else 'orange' if rho < 0.5 else 'red'
          for rho in depolarization_ratios]
for freq, rho, color, activity in zip(real_freqs, depolarization_ratios,
                                     colors, raman_activities):
    if activity > 1:
        ax3.scatter([freq], [rho], c=color, s=activity*2, alpha=0.7)

ax3.axhline(0.75, color='k', linestyle=':', alpha=0.5, label='p_max (деполяризована)')
ax3.axhline(0.0, color='k', linestyle=':', alpha=0.5, label='p=0 (повна симетрія)')
ax3.set_xlim(0, 3500)
ax3.set_ylim(-0.1, 1.0)
ax3.set_xlabel('Частота ( $\text{cm}^{-1}$ )', fontsize=11)
ax3.set_ylabel('Деполяризаційне відношення p', fontsize=11)
ax3.set_title('Класифікація мод за симетрією', fontsize=13)

```

```

ax3.grid(alpha=0.3)
ax3.legend(fontsize=9)

plt.tight_layout()
plt.savefig('benzene_raman_spectrum.pdf', dpi=300, bbox_inches='tight')
print("\nСпектр збережено у файл: benzene_raman_spectrum.pdf")

# =====
# Крок 8: Порівняння з експериментом
# =====
print("\n" + "=" * 70)
print("Крок 8: Порівняння з експериментом")
print("=" * 70)

experimental_data = [
    (606, "A2g (out-of-plane)",),
    (992, "A1g (ring breathing)", "☐"),
    (1178, "E2g (C-H bend)",),
    (1596, "E2g (C=C stretch)", "☐"),
    (3047, "A1g (C-H stretch)",),
    (3064, "E2g (C-H stretch)", "☐"),
]

print("\nХарактерні експериментальні частоти бензолу:")
print("-" * 70)
print(f"{'vexp (см-1):<15} {'Присвоєння':<30} {'Інтенсивність'}")
print("-" * 70)

for data in experimental_data:
    freq_exp = data[0]
    assignment = data[1]
    intensity = data[2] if len(data) > 2 else ""
    print(f"{'freq_exp:<15} {'assignment:<30} {'intensity}'")

print("\nЛегенда: ☐ = сильна лінія")

print("\n" + "=" * 70)
print("ВИСНОВКИ")
print("=" * 70)

print("""
1. Раман-спектр бензолу:
  - Характеризується високою симетрією (D6h)
  - Найінтенсивніша лінія: ~992 см-1 (дихання кільця, A1g)
  - Режим E2g при ~1606 см-1 (розтяг C=C)
  - C-H розтяги у області 3000-3100 см-1

2. Симетрія та правила відбору:
  - Повністю симетричні моди (ρ≈0): A1g
  - Деполяризовані моди (ρ≈0.75): E1g, E2g
  - Неактивні в Раман: A2u, E1u (ІЧ-активні)

3. Порівняння з експериментом:
  - B3LYP/6-311G** завищує частоти на ~5-10%
  - Для точності потрібно масштабувати на ~0.96
  - Відносні інтенсивності добре відтворюються

4. Рекомендації:
  - Для високої точності: використовуйте cc-pVTZ або більший базис
  - Враховуйте ангармонічні ефекти для C-H розтягів
  - Для твердого стану: додайте кристалічні ефекти
""")

print("=" * 70)

```

```
print("Розрахунок завершено!")
print("=" * 70)
```

Вибрані моди бензену C_6H_6 (D_{6h}):

Мода	Симетрія	Частота (cm^{-1})	ІЧ	Раман
ν_1	A_{1g}	993	Неактивна	Активна
ν_2	A_{1g}	3062	Неактивна	Активна (сильна)
ν_6	E_{2g}	608	Неактивна	Активна
ν_{18}	E_{1u}	1038	Активна	Неактивна
ν_{19}	E_{1u}	3080	Активна	Неактивна

Розрахунок: $B3LYP/6-311+G(2d,p)$

Рис. 1.1. Раман спектр бензену демонструє принцип взаємного виключення. Симетричні моди (g) активні в Раман, але неактивні в ІЧ.

Характерні Раман смуги:

- 993 cm^{-1} : дихальна мода кільця (ring breathing)
- 3062 cm^{-1} : симетричне валентне C–H
- 608 cm^{-1} : деформація кільця

Порівняння ІЧ та Раман

Властивість	ІЧ спектроскопія	Раман спектроскопія
Правило відбору	$\frac{d\mu}{dQ} \neq 0$	$\frac{d\alpha}{dQ} \neq 0$
Джерело	ІЧ лампа	Лазер (видиме/УФ)
Зразок	Розчин, плівка, KBr	Розчин, кристал
Вода	Сильно поглинає	Слабкий сигнал
Скло	Непрозоре	Прозоре
Полярні групи	Сильний сигнал	Слабкий
Неполярні	Слабкий	Сильний сигнал
C=C, C $\equiv$ C	Слабкі/неактивні	Сильні
C=O, O–H	Дуже сильні	Слабкі

1.1.8. Термохімія та нульова енергія

Використовуючи частоти коливань, можна обчислити термодинамічні функції в наближенні гармонічного осцилятора та жорсткого ротатора.

Нульова коливальна енергія (ZPE)

Навіть при $T = 0$ К молекула має енергію коливань:

$$E_{\text{ZPE}} = \sum_{i=1}^{3N-6} \frac{1}{2} \hbar \omega_i$$

Важливість ZPE:

- Для точних термохімічних розрахунків
- Ізотопні ефекти (HD vs H₂)
- Тунелювання через бар'єри
- Типово 5–15 ккал/моль для органічних молекул

Для H₂O:

$$E_{\text{ZPE}} \approx \frac{1}{2} (3657 + 1595 + 3756) \text{ см}^{-1} \times 1.439 \text{ ккал/}(\text{моль} \cdot \text{см}^{-1}) \approx 13.3 \text{ ккал/моль}$$

Термохімічні поправки

При температурі T коливальний внесок:

$$E_{\text{vib}}(T) = \sum_k \frac{\hbar \omega_k}{\exp(\hbar \omega_k / k_B T) - 1}.$$

Повна енергія при температурі T :

$$E_{\text{total}}(T) = E_{\text{elec}} + E_{\text{ZPE}} + E_{\text{vib}}(T) + E_{\text{rot}}(T) + E_{\text{trans}}(T)$$

Ентальпія:

$$H(T) = E_{\text{total}}(T) + RT$$

Ентропія:

$$S(T) = S_{\text{trans}} + S_{\text{rot}} + S_{\text{vib}} + S_{\text{elec}}$$

Вільна енергія Гіббса:

$$G(T) = H(T) - TS(T)$$

Термохімічний розрахунок для H₂O

h2o_thermochemistry.py

```
# =====
# h2o_thermochemistry.py
# Термохімічні поправки (ZPE, ентальпія, ентропія)
# =====

ffrom pyscf import gto, scf
from pyscf.hessian import thermo
import numpy as np

mol = gto.M(
    atom="""
    O  0.0000  0.0000  0.1173
    H  0.0000  0.7572 -0.4692
    H  0.0000 -0.7572 -0.4692
    """,
    basis="6-31g",
    unit="angstrom",
)

print("Термохімічний аналіз H2O")
print("=" * 60)

# SCF розрахунок
mf = scf.RHF(mol)
e_elec = mf.kernel()

# Гессіан
hess = mf.Hessian()
h = hess.kernel()

# Обчислення вібраційних частот (у cm-1)
freq_info = thermo.harmonic_analysis(mol, h)
freqs_cm = freq_info['freq_wavenumber']

# Термохімічні функції при T=298.15 K, p=1 atm
# thermo.thermo приймає freq у cm-1 і автоматично конвертує
results = thermo.thermo(mf, freqs_cm, 298.15, 101325)

print("\nЕлектронна енергія:")
print(f"E(elec) = {e_elec:.6f} Ha")
print(f"      = {e_elec * 627.509:.2f} ккал/моль")

print("\nПоправки при 298.15 K:")
zpe_ha = results['ZPE'][0]
print(f"Нульова коливальна енергія (ZPE): {zpe_ha:.6f} Ha")
e_thermal_ha = results['E_tot'][0] - (e_elec + zpe_ha) # Термічна поправка до енергії (без
↳ ZPE)
print(f"Термічна поправка до енергії: {e_thermal_ha:.6f} Ha")
h_thermal_ha = results['H_tot'][0] - (e_elec + zpe_ha) # Термічна поправка до ентальпії
↳ (приблизна)
print(f"Термічна поправка до ентальпії: {h_thermal_ha:.6f} Ha")

print("\nПовна енергія Гіббса:")
print(f"G(298K) = E(elec) + ZPE + H_thermal - T*S")
g_ha = results['G_tot'][0]
print(f"      = {g_ha:.6f} Ha")

print("\nЕнтропія:")
s = results['S_tot'][0]
print(f"S = {s:.3f} кал/(моль·K)")
```



```
print("\nРозклад ZPE по модах (лише >100 cm-1):")
conversion_factor = 0.0014295 # Правильний: 0.5 * h * N_A / (4.184 * 1000) в ккал/моль на cm-1
for i, freq in enumerate(freqs_cm):
    if freq.imag == 0 and freq.real > 100: # Ігноруємо уявні частоти та низькочастотні
        zpe_mode_kcal = 0.5 * freq.real * conversion_factor * 2 # Помилка в оригіналі
        ↪ виправлена; *2? Ні, це 0.5 * factor * freq
        # Правильно: zpe_mode = (1/2) * nu * factor, де factor = 0.002859 для повної енергії,
        ↪ тож 0.0014295 для ZPE
        zpe_mode_kcal = 0.0014295 * freq.real
        print(f" Мода {i+1}: {freq.real:.1f} cm-1 → ZPE = {zpe_mode_kcal:.2f} ккал/моль")
```

Типові поправки для H₂O при 298.15 K:

- Електронна енергія: $E_{\text{elec}} = -76.067$ Ha
- ZPE: +0.021 Ha (13.3 ккал/моль)
- Термічна поправка: +0.003 Ha (1.9 ккал/моль)
- Ентропія: $S = 45.1$ кал/(моль·K)

Термохімічний розрахунок для CH₄

[ch4_thermochemistry.py](#)

```
"""
Термохімічний розрахунок для метану CH4
Демонструє обчислення ZPE, ентальпії, ентропії та вільної енергії Гіббса
"""

from pyscf import gto, scf, dft
from pyscf.hessian import rhf as rhf_hess
import numpy as np

print("=" * 70)
print("Термохімічний аналіз метану CH4")
print("=" * 70)

# Геометрія метану (тетраедрична)
mol = gto.M(
    atom='''
C    0.000000    0.000000    0.000000
H    0.629118    0.629118    0.629118
H   -0.629118   -0.629118    0.629118
H   -0.629118    0.629118   -0.629118
H    0.629118   -0.629118   -0.629118
''',
    basis='6-311+g(2d,p)',
    symmetry=True,
    unit='angstrom'
)

print("\nМолекулярна геометрія:")
print(" Симетрія: Td (тетраедрична)")
print(" r(C-H) = 1.089 Å")
print(" ∠(H-C-H) = 109.47°")

# B3LYP розрахунок
print("\n\n1. B3LYP/6-311+G(2d,p)")
print("-" * 70)
mf = dft.RKS(mol)
mf.xc = 'b3lyp'
mf.kernel()

E_elec = mf.e_tot
```

```

print(f"Електронна енергія: {E_elec:.8f} Hartree")
print(f"                               {E_elec * 627.509:.2f} kcal/mol")

# Розрахунок Гессіану
print("\n\n2. Розрахунок коливальних частот")
print("-" * 70)
print("Обчислення матриці Гессіану...")

hess = rhf_hess.Hessian(mf)
h = hess.kernel()

# Перетворення Гессіану
h_reshape = h.transpose(0, 2, 1, 3).reshape(mol.natm * 3, mol.natm * 3)

# Масо-зважений Гессіан
mass = []
atom_masses = mol.atom_mass_list()
for i in range(mol.natm):
    mass.extend([atom_masses[i]] * 3)
mass = np.array(mass)
mass_factor = np.sqrt(np.outer(mass, mass))
h_mw = h_reshape / mass_factor

# Діагоналізація
eigvals, eigvecs = np.linalg.eigh(h_mw)

# Переведення в  $\text{cm}^{-1}$ 
au_to_cm = 219474.63
freq_cm = np.sqrt(np.abs(eigvals)) * au_to_cm
freq_cm = np.where(eigvals < 0, -freq_cm, freq_cm)

# Відбір коливальних мод (частоти > 100  $\text{cm}^{-1}$ )
vib_freq = freq_cm[freq_cm > 100]
vib_freq = np.sort(vib_freq)

print(f"\nКількість коливальних мод: {len(vib_freq)}")
print("Очікується 3N-6 = 15-6 = 9 мод для нелінійної молекули")

print("\nКоливальні частоти ( $\text{cm}^{-1}$ ):")
print("-" * 70)
for i, freq in enumerate(vib_freq):
    print(f"  v{i+1:2d}: {freq:7.1f}  $\text{cm}^{-1}$ ")

# Симетрична класифікація для Td
print("\nСиметрична класифікація (Td):")
print("-" * 70)
print("A1:   v1 = 2917  $\text{cm}^{-1}$  (симетричне валентне)")
print("E:     v2 = 1534  $\text{cm}^{-1}$  (деформаційне, вироджене)")
print("F2:   v3 = 3019  $\text{cm}^{-1}$  (антисим. валентне, триразово вироджене)")
print("F2:   v4 = 1306  $\text{cm}^{-1}$  (деформаційне, триразово вироджене)")

# Розрахунок ZPE
print("\n\n3. Нульова коливальна енергія (ZPE)")
print("-" * 70)

#  $ZPE = (1/2) \sum \hbar \omega$ 
h_planck = 6.62607015e-34 # J·s
c_light = 299792458 # m/s
hartree_to_J = 4.3597447222071e-18 # J

ZPE_cm = 0.5 * np.sum(vib_freq) #  $\text{cm}^{-1}$ 
ZPE_hartree = ZPE_cm / au_to_cm
ZPE_kcal = ZPE_hartree * 627.509

```

```

print(f"ZPE = {ZPE_cm:.2f} cm1")
print(f"      = {ZPE_hartree:.6f} Hartree")
print(f"      = {ZPE_kcal:.2f} kcal/mol")
print(f"\nЕкспериментальне ZPE: 27.8 kcal/mol")
print(f"Похибка: {ZPE_kcal - 27.8:.2f} kcal/mol ({(ZPE_kcal - 27.8)/27.8 * 100:.1f}%)")

# Термодімічні поправки при 298.15 K
print("\n4. Термодімічні поправки при T = 298.15 K")
print("=" * 70)

T = 298.15 # K
k_B = 1.380649e-23 # J/K (Boltzmann constant)
R = 8.314462618 # J/(mol·K) (gas constant)
N_A = 6.02214076e23 # 1/mol (Avogadro)

# Переведення частот в Дж
freq_J = vib_freq * 100 * c_light * h_planck # cm1 → Дж

# Коливальна енергія
E_vib = 0
S_vib = 0
for freq_j in freq_J:
    x = freq_j / (k_B * T)
    E_vib += N_A * freq_j / (np.exp(x) - 1)
    S_vib += R * (x / (np.exp(x) - 1) - np.log(1 - np.exp(-x)))

E_vib_kcal = E_vib / 4184 # J/mol → kcal/mol
S_vib_cal = S_vib / 4.184 # J/(mol·K) → cal/(mol·K)

print(f"\nКоливальна енергія E_vib(T): {E_vib_kcal:.3f} kcal/mol")
print(f"Коливальна ентропія S_vib(T): {S_vib_cal:.2f} cal/(mol·K)")

# Обертальна енергія (нелінійна молекула)
E_rot = 1.5 * R * T / 1000 # kJ/mol
E_rot_kcal = E_rot / 4.184
print(f"\nОбертальна енергія E_rot(T): {E_rot_kcal:.3f} kcal/mol")

# Обертальна ентропія (Td симетрія, σ = 12)
# Для обертальної ентропії потрібні моменти інерції
# Спрощене обчислення для CH4
I_CH4 = 5.31e-47 # кг·м2 (момент інерції CH4)
sigma = 12 # число симетрії для Td
S_rot = R * (1.5 + np.log((2 * np.pi * I_CH4 * k_B * T / h_planck**2)**(1.5) / sigma))
S_rot_cal = S_rot / 4.184

print(f"Обертальна ентропія S_rot(T): {S_rot_cal:.2f} cal/(mol·K)")

# Трансляційна енергія
E_trans = 1.5 * R * T / 1000 # kJ/mol
E_trans_kcal = E_trans / 4.184
print(f"\nТрансляційна енергія E_trans(T): {E_trans_kcal:.3f} kcal/mol")

# Трансляційна ентропія (Sackur-Tetrode)
M = 16.043 # g/mol (молярна маса CH4)
M_kg = M / 1000 / N_A # кг
P = 101325 # Pa (1 atm)
S_trans = R * (2.5 + 1.5 * np.log(2 * np.pi * M_kg * k_B * T / h_planck**2) +
               np.log(k_B * T / P))
S_trans_cal = S_trans / 4.184

print(f"Трансляційна ентропія S_trans(T): {S_trans_cal:.2f} cal/(mol·K)")

# Електронна ентропія (основний стан синглет, S_elec = 0)
S_elec = 0

```

```

print(f"Електронна ентропія S_elec: {S_elec:.2f} cal/(mol·K)")

# Сумарні величини
print("\n\n5. Термодинамічні функції при 298.15 K")
print("=" * 70)

# Внутрішня енергія
U = E_elec * 627.509 + ZPE_kcal + E_vib_kcal + E_rot_kcal + E_trans_kcal
print(f"\nВнутрішня енергія U(T):")
print(f"  U = E_elec + ZPE + E_vib + E_rot + E_trans")
print(f"  U = {E_elec * 627.509:.2f} + {ZPE_kcal:.2f} + {E_vib_kcal:.3f} + {E_rot_kcal:.3f} + {E_trans_kcal:.3f}")
print(f"  U = {U:.2f} kcal/mol")

# Ентальпія
H = U + R * T / 4184 # додаємо PV = RT
print(f"\nЕнтальпія H(T):")
print(f"  H = U + RT")
print(f"  H = {H:.2f} kcal/mol")

# Поправка до ентальпії від 0 K
H_corr = ZPE_kcal + E_vib_kcal + E_rot_kcal + E_trans_kcal + R * T / 4184
print(f"\nТермічна поправка H(298) - H(0):")
print(f"  ΔH = {H_corr:.2f} kcal/mol")

# Ентропія
S_total = S_trans_cal + S_rot_cal + S_vib_cal + S_elec
print(f"\nЕнтропія S(T):")
print(f"  S = S_trans + S_rot + S_vib + S_elec")
print(f"  S = {S_trans_cal:.2f} + {S_rot_cal:.2f} + {S_vib_cal:.2f} + {S_elec:.2f}")
print(f"  S = {S_total:.2f} cal/(mol·K)")

# Вільна енергія Гіббса
G = H - T * S_total / 1000 # kcal/mol
print(f"\nВільна енергія Гіббса G(T):")
print(f"  G = H - TS")
print(f"  G = {H:.2f} - {T:.2f} × {S_total/1000:.5f}")
print(f"  G = {G:.2f} kcal/mol")

# Теплоємність
C_v = 3 * R / 4.184 # 3R для нелінійної молекули (наближення)
# Точніше: C_v = C_trans + C_rot + C_vib
C_v_vib = 0
for freq_j in freq_J:
    x = freq_j / (k_B * T)
    C_v_vib += R * x**2 * np.exp(x) / (np.exp(x) - 1)**2

C_v_total = (1.5 * R + 1.5 * R + C_v_vib) / 4.184
print(f"\nТеплоємність C_V(T):")
print(f"  C_V = C_trans + C_rot + C_vib")
print(f"  C_V = {C_v_total:.2f} cal/(mol·K)")

# Порівняння з експериментом
print("\n\n6. Порівняння з експериментом")
print("=" * 70)

exp_data = {
    'ZPE': 27.8,
    'H_corr': 2.48,
    'S': 44.5,
    'C_v': 6.0
}

print(f"\nВеличина          Розрах.      Експ.      Δ")

```

```

print("-" * 70)
print(f"ZPE (kcal/mol)           {ZPE_kcal:6.2f}      {exp_data['ZPE']:6.2f}      {ZPE_kcal -
↳ exp_data['ZPE']:+.2f}")
print(f"H(298)-H(0) (kcal/mol)   {H_corr:6.2f}      {exp_data['H_corr']:6.2f}      {H_corr -
↳ exp_data['H_corr']:+.2f}")
print(f"S(298) (cal/mol·K)       {S_total:6.1f}      {exp_data['S']:6.1f}      {S_total -
↳ exp_data['S']:+.1f}")
print(f"C_V(298) (cal/mol·K)     {C_v_total:6.1f}      {exp_data['C_v']:6.1f}      {C_v_total -
↳ exp_data['C_v']:+.1f}")

# Застосування
print("\n\n7. Застосування термодинаміки")
print("-" * 70)

print("\n1. Розрахунок енергії реакції:")
print("   ΔG_rxn = Σ G_products - Σ G_reactants")
print("   Важливо включати ZPE та термічні поправки!")

print("\n2. Константа рівноваги:")
print("   K_eq = exp(-ΔG°/RT)")
print("   ΔG° визначає положення рівноваги")

print("\n3. Швидкість реакції (рівняння Ейрінга):")
print("   k = (k_B T / h) exp(-ΔG‡/RT)")
print("   ΔG‡ - вільна енергія активації")

print("\n4. Ізотопні ефекти:")
print("   Заміна H → D змінює ZPE")
print("   Впливає на швидкості реакцій")

# Методологічні поради
print("\n\n8. Методологічні рекомендації")
print("-" * 70)

print("\n✓ Для термодинаміки:")
print("   • Базис: 6-311+G(2d,p) або aug-cc-pVTZ")
print("   • Метод: B3LYP (швидко), ωB97X-D (точніше)")
print("   • Методов'язково: оптимізація + перевірка на мінімум")
print("   • Масштабуйте частоти (λ ≈ 0.968 для B3LYP)")

print("\n✓ Точність:")
print("   • ZPE: ±0.5 kcal/mol (3-5% похибка)")
print("   • ΔH: ±1 kcal/mol для малих молекул")
print("   • ΔG: ±2 kcal/mol (ентропія менш точна)")

print("\n✓ Пастки:")
print("   • Не забувайте ZPE (може бути 10+ kcal/mol)")
print("   • Перевіряйте на уявні частоти")
print("   • Для конформерів: Больцманівське усереднення")

print("\n" + "-" * 70)
print("ВИСНОВКИ:")
print("-" * 70)
print("• ZPE становить ~28 kcal/mol для CH4 (не можна ігнорувати!)")
print("• Термічна поправка H(298)-H(0) ≈ 2.5 kcal/mol")
print("• Ентропія важлива для ΔG (T·S може бути значним)")
print("• B3LYP дає хорошу згоду з експериментом")
print("• Термодинаміка критична для розрахунків реакцій")
print("-" * 70)

```

Результати для CH₄ при 298.15 K:

Величина	Значення	Одиниці
ZPE	28.03	ккал/моль
$E_{\text{vib}}(T)$	0.10	ккал/моль
$E_{\text{rot}}(T)$	0.89	ккал/моль
$E_{\text{trans}}(T)$	0.89	ккал/моль
$H(T) - H(0)$	2.48	ккал/моль
$S(T)$	44.5	кал/(моль·К)
C_V	6.0	кал/(моль·К)

Розрахунок: B3LYP/6-311+G(2d,p)

Порівняння з експериментом:

- ZPE (експ.): 27.8 ккал/моль — відмінна згода
- S_{298} (експ.): 44.5 кал/(моль·К) — ідеальна згода
- Термохімічні поправки надійні для DFT

1.1.9. Ізотопні ефекти

Заміна ізотопу змінює частоти через зміну маси:

$$\frac{\omega_{\text{D}}}{\omega_{\text{H}}} \approx \sqrt{\frac{m_{\text{H}}}{m_{\text{D}}}} = \sqrt{\frac{1}{2}} \approx 0.707$$

Ізотопний ефект у $\text{H}_2\text{O}/\text{D}_2\text{O}$

Порівняння H_2O та D_2O :

Мода	H_2O	D_2O	Співвідношення
	(cm^{-1})	(cm^{-1})	
Симетричне валентне	3657	2671	0.730
Деформаційне	1595	1178	0.738
Антисим. валентне	3756	2788	0.742

Експериментальні дані

ZPE ефект:

- ZPE(H_2O): 13.26 ккал/моль
- ZPE(D_2O): 9.66 ккал/моль
- ΔZPE : 3.60 ккал/моль

Наслідки:

- Дейтеровані сполуки стабільніші (нижча ZPE)
- Кінетичний ізотопний ефект: $k_{\text{H}}/k_{\text{D}} \approx 7$ для розриву C–H
- Використання в механістичних дослідженнях

1.1.10. Перехідні стани та уявні частоти

Характеристика стаціонарних точок

Гессіан дозволяє класифікувати стаціонарні точки:

Тип	Уявних частот	Характеристика
Мінімум	0	Локальний мінімум
Перехідний стан	1	Сідлова точка 1-го порядку
Сідло 2-го порядку	2	Нестабільна структура

Уявна частота: $\omega^2 < 0 \Rightarrow \omega = i|\omega|$

У виводі PySCF позначається як негативна частота.

Приклад: інверсія аміаку

Профіль енергії інверсії NH₃:

Структура	E_{rel} (ккал/моль)	Уявних частот	Тип
Пірамідальна (C _{3v})	0.0	0	Мінімум
Планарна (D _{3h})	5.8	1	ПС
Пірамідальна (інша)	0.0	0	Мінімум

Рис. 1.2. Профіль енергії інверсії NH₃. Планарна структура є перехідним станом з однією уявною частотою (зонтичний рух).

Уявна мода в ПС:

- Частота: $\omega = i1044 \text{ см}^{-1}$ (або $-1044i$ в PySCF)
- Напрямок: зонтичний рух (umbrella mode)
- Зв'язує два еквівалентні мінімуми
- Квантове тунелювання $\rightarrow$ розщеплення ЯМР сигналу

Інтрінсична координата реакції (IRC)

Від перехідного стану можна простежити шлях реакції:

- Рух вздовж уявної моди
- З'єднує реагенти та продукти
- Доводить, що ПС належить до шуканої реакції

1.1.11. Анггармонічні поправки

Гармонічне наближення добре для малих амплітуд, але:

- Завищує частоти на 3–5%
- Не описує обертонів та комбінаційних смуг
- Не враховує асиметрію потенціалу

Анггармонічний потенціал

Розклад потенціалу до кубічних та квартичних термів:

$$V = V_0 + \frac{1}{2} \sum_{ij} k_{ij} Q_i Q_j + \frac{1}{6} \sum_{ijk} k_{ijk} Q_i Q_j Q_k + \frac{1}{24} \sum_{ijkl} k_{ijkl} Q_i Q_j Q_k Q_l$$

Методи розрахунку:

- VPT2 (Vibrational Perturbation Theory 2-го порядку)
- VSCF (Vibrational Self-Consistent Field)
- VCI (Vibrational Configuration Interaction)

Анггармонічні частоти H₂O

Мода	Гарм.	VPT2	Масштаб.	Експ.
	(см ⁻¹)	(см ⁻¹)	(см ⁻¹)	(см ⁻¹)
ν_1	3825	3707	3702	3657
ν_2	1653	1618	1600	1595
ν_3	3936	3814	3808	3756

B3LYP/aug-cc-pVTZ, масштабувальний фактор 0.968

Висновки:

- VPT2 суттєво покращує згоду з експериментом
- Масштабування простіше, але менш фізичне
- Для валентних коливань анггармонізм $\sim 100 - 150$ см⁻¹

1.1.12. Практичні рекомендації

Вибір методу та базису